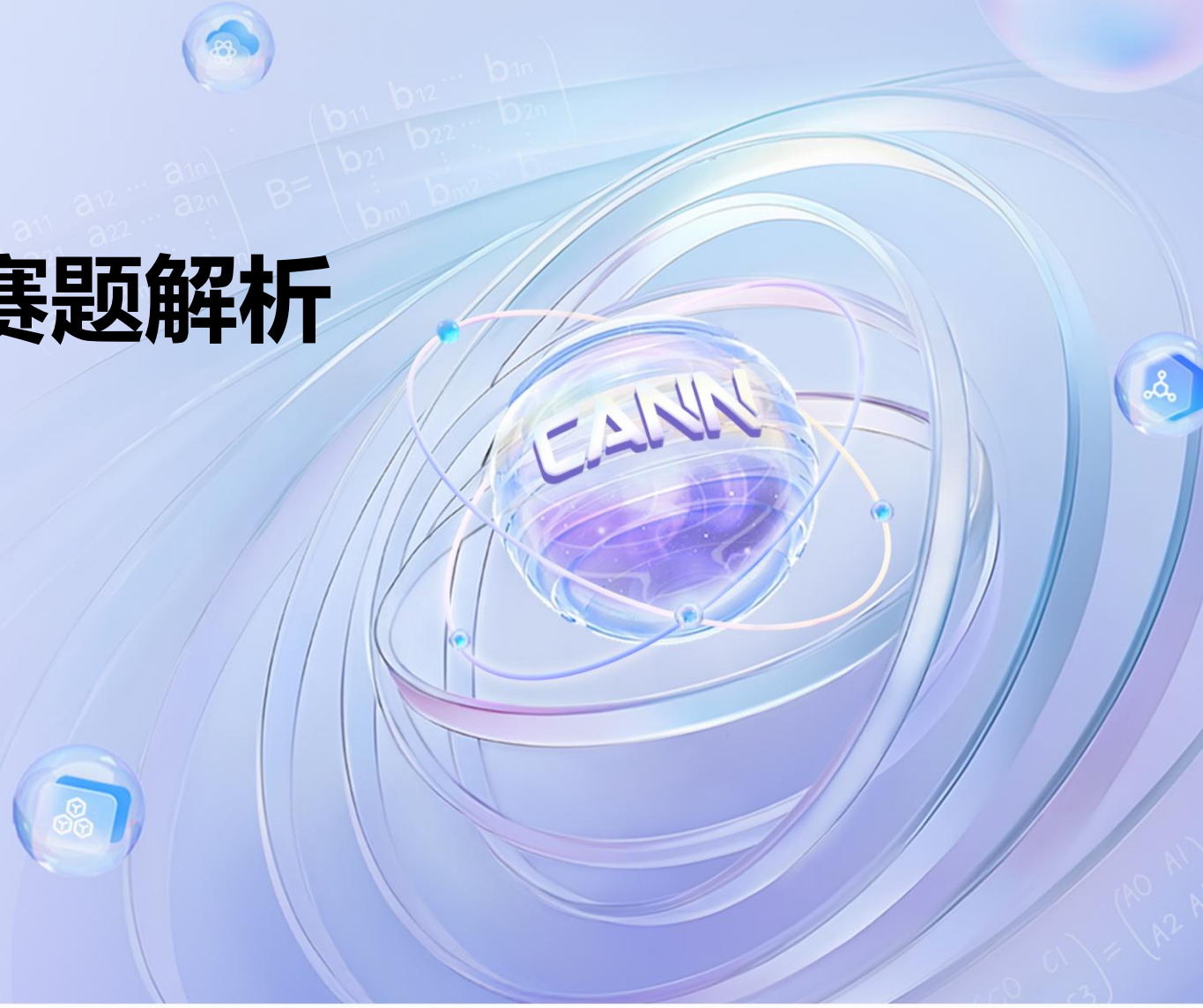
CANN 挑战赛决赛赛题解析

分享嘉宾：起光

时间：2026年11月



CANN

目录

Part 1 赛程安排

Part 2 赛题解读

Part 3 交流答疑

大赛简介及日程

CANN挑战赛是由 CANN开源社区 主办的赛事。大赛依托华为算法挑战赛，围绕昇腾AI算子开发、功能实现与性能优化设置竞赛任务，并提供配套学习资料和在线实践环境，帮助参赛选手了解算子开发流程，提升编程实践和性能优化能力。本届赛事设置 **杭厦、上合、京津东北、西北、西南**五大赛区，面向全国相关区域高校学生开放报名。欢迎高校学子参与挑战。

当前进度

初赛

在线答题

7/21	9/5	10/16	10/17 (周六)	10/18	10/19
启动报名	赛题发布 启动初赛	报名截止	线上初赛截止	公示	晋级名单公布

前32强队伍晋级复赛

决赛

第一批 | **京津东北 · 西南 · 西北**

10/23*	11/6	11/7（周六）
赛题发布 + 直播解读	外地选手到场	现场决赛&颁奖

第二批 | **上合 · 杭厦**

10/30	11/13	11/14（周六）
赛题发布 + 直播解读	外地选手到场	现场决赛&颁奖

*首批赛题发布10/23；本地选手比赛当天到场

目录

Part 1 赛程安排

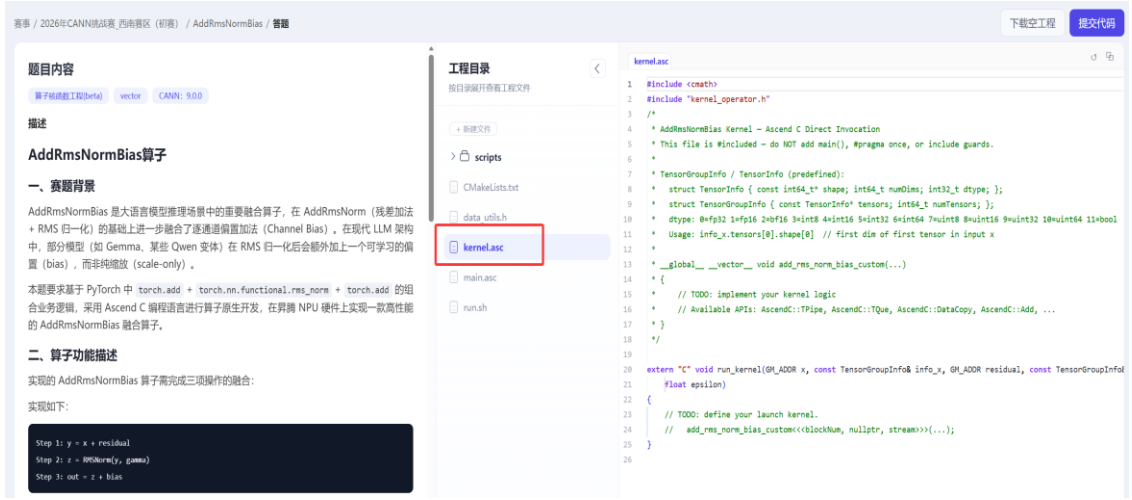
Part 2 赛题解读
（京津东北、西北、西南）

Part 3 交流答疑

赛题解读-判题过程与得分介绍

判题过程介绍

- 完成赛题要求的一个文件并点击提交代码。



- 平台会自动跳转到提交界面，后台Case验证完成后返回结果。



注意：编译错误可在执行结果界面查看具体报错

得分规则

- 提交完成后可查看提交排名。



- 本次比赛共15个测试点，所有case点精度全部通过才会计分。
- 每个测试点单独计分，逻辑如下（T为最优性能，t为当前提交性能）：

$$\frac{100}{1+\log_{1.5}\frac{t}{T}}$$

- 排行榜显示的最终分数为所有case得分的均值。若得分计算一致，则以提交时间进行排序，提交越早，排名越高。

注意：排行榜仅针对最后一次提交呈现，请确保比赛结束前提交的是最优实现

CANN

（1/4）京津东北决赛赛题解读-赛题背景

■ 赛题背景：

① 视频生成大模型推理时，会把一批视频按帧打包成一条长序列送进 attention：每帧的 token 数由该帧分辨率决定（动态分辨率 / 多镜头混跑），每帧时间戳由采集时间决定（可变帧率下帧间隔不相等），且帧在张量中的存储顺序未必等于时间顺序（稀疏采样、关键帧优先、多镜头时间轴重叠）。

② 因此每个 token 的位置坐标已无法从下标直接算出：要先在“每帧 token 数”表上定位所属存储帧，再按该帧宽高反解行列，最后按存储帧的位置索引取出时间戳。

③ 这就是 n-d RoPE 的变长 / 变帧率形态（VarLen ND RoPE）：位置编码三个轴的坐标全部由输入数据决定。

$$\text{start}[j] = \Sigma\, n[k]\ (k<j);\quad \text{time}[j] = \Sigma\, \Delta[k]\ (k<j)$$
$$j = \text{searchsorted}(\text{start}, s, \text{right=True}) - 1;\quad \text{idx} = s - \text{start}[j];\quad t = \text{time}[\text{frame_pos}[j]]$$
$$w = \text{idx} \bmod W_j;\quad h = \text{idx} \operatorname{div} W_j;\quad \varphi_a[i] = \text{coord_a} \cdot \text{theta}^{\wedge}(-2i/d_a)$$
$$y[2m] = x[2m]\cdot\cos\varphi - x[2m+1]\cdot\sin\varphi;\quad y[2m+1] = x[2m+1]\cdot\cos\varphi + x[2m]\cdot\sin\varphi$$

■ 赛题特点

① 本次比赛要求使用 Ascend C 编程语言完成 VarLenNdRotaryPosEmb 算子开发

② 在满足精度的基础上，比拼算子性能：帧定位、数据依赖索引、FP32 三角函数

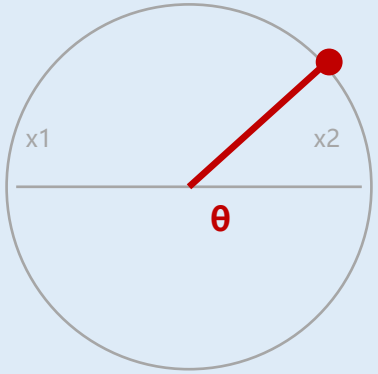
核心要点

- 帧定位：frame_tokens 排他前缀和
- 时基：frame_deltas 前缀和 + frame_pos 查表
- 帧内坐标：按该帧 W_j 反解 (h, w)
- 三轴频率 + FP32 cos/sin，相邻对旋转

CANN

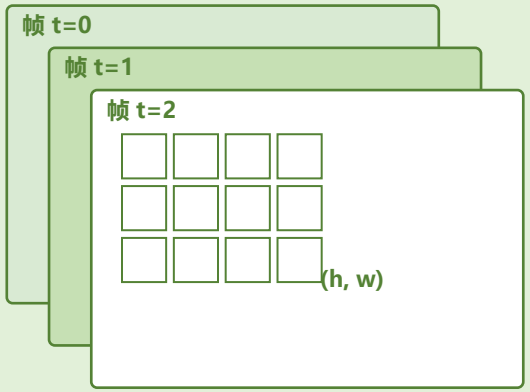
(2/4)京津东北决赛赛题解读-算子来源：从 RoPE 到 3D RoPE

① 标准 RoPE：相邻维度看成平面



位置 pos 决定旋转角：θ = pos × inv_freq
每对 (x1, x2) 做正交旋转：
y1 = x1·cosθ − x2·sinθ
y2 = x2·cosθ + x1·sinθ

② 视频 → 3D 坐标 (t, h, w)



视频按帧打包成一条长序列；
每个 token 属于某帧：时间 t 来自时间戳，
行 h / 列 w 来自该帧内的网格。

③ D 维按三轴分段



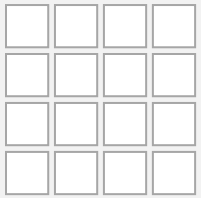
- 每段用对应轴坐标 × 自己的 inv_freq
- 段内相邻对 (2m, 2m+1) 交错复制 cos/sin
- 三段互不跨段配对
- d_t + d_h + d_w = D（各段为偶数）

关键区别：3D RoPE 不是把整条 D 维旋转同一个角度，而是把 D 维切成 [d_t | d_h | d_w] 三段，分别用 (t, h, w) 生成角度，再在段内做相邻对旋转。

CANN

(3/4)京津东北决赛赛题解读-变长/变帧率：坐标为何不能直接算

初赛：定长网格（下标可直接算）



H、W 全局固定
(h, w) = (idx // W, idx % W)
pos 单调；时间序 = 存储序

决赛：变长 / 变帧率（坐标由数据决定）



n_j、W_j 逐帧不同
time 由 frame_pos 查表
存储序 ≠ 时间序

① 帧定位
frame_tokens 排他前缀和 → starts
searchsorted(starts, s) − 1 → 帧 j

② 时间基准
frame_deltas 排他前缀和 → times
t = times[frame_pos[j]]

③ 帧内坐标
idx = s − starts[j]
w = idx % W_j；h = idx // W_j

④ 三轴旋转
angle = coord × inv_freq
cos/sin → 相邻对旋转

三个“不能假设”

- 不能假设全局统一的 H/W：帧内反解的除数是数据依赖的 W_j
- 不能假设 t 单调：frame_pos 可乱序、可重复，多个存储帧可共享同一时间戳
- 不能假设帧序 = 时间序：存储顺序与时间顺序解耦

CANN

(4/4)京津东北决赛赛题解读-题目概述：VarLenNdRotaryPosEmb算子开发

■ 题目概述：

实现的 VarLenNdRotaryPosEmb 算子需完成：帧定位（frame_tokens 前缀和）→ 时间基准（frame_deltas 前缀和 + frame_pos 查表）→ 帧内坐标反解 → 三轴频率与 cos/sin → 按 [d_t | d_h | d_w] 分段相邻对旋转，输出与输入同 shape 同 dtype。

类型	参数名	类型	维度形状	支持数据类型	数据格式	备注
INPUT（必选）	x	tensor	(S, N, D)	float16, float32	ND	待旋转张量，S=Σn_j
INPUT（必选）	frame_tokens	tensor	(F,)	int32	ND	每帧 token 数 n_j=H_j·W_j
INPUT（必选）	frame_widths	tensor	(F,)	int32	ND	每帧宽 W_j，须整除 n_j
INPUT（必选）	frame_deltas	tensor	(F,)	float32	ND	帧时长，时基为排他前缀和
INPUT（必选）	frame_pos	tensor	(F,)	int32	ND	存储帧在时基中的位置索引
ATTR（属性）	rope_dim_list	list_int	[3]	-	-	三轴分段 [d_t, d_h, d_w]
ATTR（属性）	theta	float	-	-	-	基础频率基数，默认 10000.0
OUTPUT（输出）	y	tensor	(S, N, D)	与 x 一致	ND	旋转位置编码结果

■ 算子功能分解（小算子拼接）：

① 帧定位与时基
前缀和 / searchsorted
→
② 帧内坐标 + 三轴频率
mod / div / inv_freq（FP32）
→
③ 分段相邻对旋转
cos / sin / mul / add
→
④ 输出 y
形状、类型与 x 一致

■ 约束说明：

芯片类型：Atlas A2（910B）
维度范围：3 维输入（S, N, D）；S∈[1,262144]，F∈[1,128]，N∈[1,128]，D∈[8,4096] 且为偶数
帧约束：n_j∈[2,65536]，W_j∈[1,n_j] 且 n_j % W_j==0；d_t+d_h+d_w=D（各段偶数）
索引语义：frame_pos 可重复、可乱序，存储序与时间序解耦
精度与对齐：前缀和 / 坐标 / inv_freq / cos/sin 全程 FP32；N·D 非 32B 对齐与尾块需正确处理

CANN

(1/2) 西北决赛赛题解读-赛题背景

■ 赛题背景：

① Mixture-of-Experts（MoE）是当前大语言模型的主流扩展架构：模型由大量专家子网络构成，每个 token 在推理时只激活其中少量专家，在保持大参数容量的同时控制推理成本。

② DeepSeek V3 的 MoE 路由在 top-k 之上增加一层“分组受限”结构：全部专家划分为若干 group，每个 token 先从 group 中选出一部分，再在选中的 group 内选专家，且每 token 只能从有限的 group 中选取。

③ 这种分层受限选择在保证专家负载均衡的同时，比全量 top-k 更有结构；评分使用 sigmoid，expert_bias 仅参与选择、不进入最终权重。

$$s[t,e] = \mathrm{sigmoid}(x[t,e]);\quad sel[t,e] = s[t,e] + bias[e]$$
$$gscore[t,g] = \Sigma\ sel[t,e]\ (e \in \mathrm{top2\ of\ group}\ g)$$
$$groups = \mathrm{top}(G_lim, gscore);\quad ids = \text{top-k(sel masked to selected groups)}$$
$$w[t,j] = s[t,ids[t,j]] \,/\, \Sigma\ s[t,ids[t,j']] \times route_scale$$

核心要点

- sigmoid 评分（FP32），bias 仅参与选择
- 组分数 = 组内 sel 的 top-2 之和（sel = sigmoid + bias）
- 每 token 选 top G_lim 组，再组内选 top-k
- 权重归一化 × route_scale，输出 ids / weights

■ 赛题特点

① 本次比赛要求使用 Ascend C 编程语言完成 GroupedTopkMoe 算子开发

② 在满足精度的基础上，比拼算子性能：稳定 sigmoid、分组 top-2、受限 top-k 与 tie 规则

CANN

（2/2）西北决赛赛题解读-题目概述：GroupedTopkMoe算子开发 (2/2)

■ 题目概述：

实现的 GroupedTopkMoe 算子需完成：对 router_logits 逐元素 sigmoid 评分 → 专家按连续区间分组、组内 top-2 求和得组分数 → 每 token 选 top G_lim 个组 → 仅在选中组内按 sel 选 top-k 专家 → 对选中专家的 sigmoid 分数归一化并乘 route_scale，输出 topk_weights 与 topk_ids。

类型	参数名	类型	维度形状	支持数据类型	数据格式	备注
INPUT（必选）	router_logits	tensor	(T, E)	float16, float32	ND	路由 logits
INPUT（可选）	expert_bias	tensor	(E,)	float32	ND	负载均衡偏置，仅参与选择
ATTR（属性）	n_expert_groups	int	-	-	-	分组数 G，默认 8
ATTR（属性）	n_limited_groups	int	-	-	-	每 token 最多选组数 G_lim，默认 3
ATTR（属性）	router_top_k	int	-	-	-	每 token 选中专家数 k，默认 8
ATTR（属性）	route_scale	float	-	-	-	归一化缩放系数，默认 2.5
OUTPUT（输出）	topk_weights	tensor	(T, k)	与 logits 一致	ND	选中专家的归一化权重
OUTPUT（输出）	topk_ids	tensor	(T, k)	int32	ND	选中专家 id

■ 算子功能分解（小算子拼接）：

① sigmoid 评分
σ(x)（FP32）

→

② 分组 top-2 求和
连续分组 / top-2

→

③ 受限选组 + 组内 top-k
group mask / 降序 top-k

→

④ 权重归一化
sum / div × route_scale

■ 约束说明：

芯片类型：Atlas A2（910B）
维度范围：2 维输入（T, E）；T∈[1,262144]，E∈[64,8192] 且能被 G 整除
属性约束：G∈[2,16]，G_lim∈[1,G]，k∈[1,E]，且 G_lim × (E/G) ≥ k
Tie 规则：组分数并列取组索引升序；专家分数并列取专家索引升序
精度要求：sigmoid 必须 FP32，避免大 logits 溢出；topk_ids 需与参考实现完全一致

CANN

（1/2）西南决赛赛题解读-赛题背景

■ 赛题背景：

① DeepSeek V4 用 Hyper-Connections（HC）替代传统残差连接：每个 block 维护 hc_mult 份 hidden state 副本，block 间通过一组混合权重完成“塌缩-展开”传递。

② 其核心约束是 Manifold-Constrained Hyper-Connections（mHC）：把副本间的组合矩阵通过 Sinkhorn-Knopp 迭代归一化，投影到双随机矩阵流形上，保证信号在深层堆叠中非扩张传播。

③ Sinkhorn-Knopp 是运筹学与最优传输中的经典迭代算法：通过交替的按行 / 按列归一化，把非负矩阵推近双随机矩阵。

$$
\begin{aligned}
&\text{pre} = \text{sigmoid}(z_pre) + \varepsilon;\quad \text{post} = 2\cdot\text{sigmoid}(z_post)\\
&A[i,j] = \text{softmax}(z_comb, \text{dim}=-1)[i,j] + \varepsilon\\
&M^{(1)}[i,j] = A[i,j] \,/\, (\Sigma\, A[i',j] + \varepsilon)\\
&\text{for } t = 2..\text{iters}:\quad M[i,j] \leftarrow M[i,j] \,/\, (\Sigma\, M[i,j'] + \varepsilon);\quad M[i,j] \leftarrow M[i,j] \,/\, (\Sigma\, M[i',j] + \varepsilon)\\
&\text{comb} = M^{(\text{iters})}
\end{aligned}
$$

核心要点

- 三段切分 [H | H | H×H]
- 门控：pre = σ + ε，post = 2σ；comb 行 softmax + ε
- 列 → (行 → 列) × (iters-1) 交替归一化
- 输出第 iters 轮轨迹状态（非收敛极限）

■ 赛题特点

① 本次比赛要求使用 Ascend C 编程语言完成 SinkhornNorm 算子开发

② 在满足精度的基础上，比拼算子性能：softmax 数值稳定、固定轮数迭代轨迹、行/列归约

CANN

(2/2)西南决赛赛题解读-题目概述：SinkhornNorm算子开发

■ 题目概述：

实现的 SinkhornNorm 算子需完成：将 mixes 最后一维按 [H | H | H×H] 三段切分 → pre=sigmoid(z_pre)+ε、post=2×sigmoid(z_post) → comb 按行 softmax 后加 ε → Sinkhorn-Knopp 迭代（先列归一化 1 次，再按“行→列”重复 iters-1 次，分母均 +ε）→ 输出 pre、post、comb。

类型	参数名	类型	维度形状	支持数据类型	数据格式	备注
INPUT（必选）	mixes	tensor	(B, S, mix_hc)	float32	ND	混合 logits，mix_hc=(2+H)×H
ATTR（属性）	hc_mult	int	-	-	-	副本数 H，默认 4
ATTR（属性）	hc_sinkhorn_iters	int	-	-	-	迭代轮数，默认 20
ATTR（属性）	eps	float	-	-	-	数值稳定常数，默认 1e-6
OUTPUT（输出）	pre	tensor	(B, S, H)	float32	ND	塌缩权重
OUTPUT（输出）	post	tensor	(B, S, H)	float32	ND	展开权重，值域 (0,2)
OUTPUT（输出）	comb	tensor	(B, S, H, H)	float32	ND	双随机组合矩阵

■ 算子功能分解（小算子拼接）：

① 三段切分 + 门控
split / sigmoid
→
② comb 行 softmax
softmax(-1) + ε
→
③ Sinkhorn 迭代
列 → (行→列)×(iters-1)
→
④ 输出 pre / post / comb
全程 FP32

■ 约束说明：

芯片类型：Atlas A2（910B）；输入输出及中间计算全程 FP32
维度范围：3 维输入（B, S, mix_hc）；B∈[1,128]，S∈[1,8192]，H∈[2,128]
形状约束：mix_hc=(2+H)×H；iters∈[1,32]
迭代语义：先列归一化 1 次，再按“行→列”重复 iters-1 次；次序不可交换
输出：第 iters 轮的精确轨迹状态，不是收敛极限

CANN

赛事规则解读

① Host / Kernel 分工：Host 仅负责参数校验、shape 推导、tiling 逻辑和启动配置；Kernel 负责完成完整计算流程。
示例：Host 只计算 tiling / block 数；若 Host 先算好部分结果再传给 Kernel，即违规。

#	禁止行为（12 条）	示例说明
1	Host 侧读取输入张量进行计算	Host 直接计算x+residual 再传回 Device
2	Host 侧对输入数据进行重排	Host 先按测试形状重排 token/通道，再 H2D 给 Kernel
3	Host 侧生成预查表，传入 Kernel 直接取值	Host 预算 cos/sin 或 top-k 索引表，Kernel 只查表
4	Host 侧完成向量/矩阵计算，传入 Kernel 使用	Host 先算好 RMS 系数/分数/矩阵乘/逆频率，Kernel 直接用
5	硬编码、按测试用例或数据规律跳算	按固定 shape/case_id 直接返回答案，或识别数据规律后跳算
6	写入标记跳过计算	Kernel 写 flag/标记，后续轮次看到标记就不再计算
7	利用旧输出或未初始化内存跳过计算	复用上一次的 y 或未初始化 workspace 当作本次结果
8	逐条匹配评测用例编写专属实现	对 (B,S,D) 精确元组写单独分支/单独 kernel
9	修改或干扰判题、精度校验和计时	改计时区间、绕过精度比较或篡改判题结果
10	注入空 kernel、识别预热轮次或把计算移出计时	warmup 轮不干活、正式计时才算，或在其他地方计算，注入空 kernel占位等。
11	在多轮次的判题中加缓存来减少耗时	第 1 轮算完缓存结果，后续同输入直接返回缓存
12	一次计算通过多次 kernel 来完成计算	把一次完整的计算过程拆成多个 kernel 来完成

③ **最终核验：**比赛结束后进行人工核验和查重；未按规则提交的代码，官方将按违规代码作废成绩。

CANN

目录

Part 1 赛程安排

Part 2 赛题解读

Part 3 交流答疑

交流答疑

讨论贴

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://atomgit.com/cann